

Lab 1: ArrayList time comparison and String manipulation

Due: 11:59pm on Mar 18, 2025 (Tuesday)

In Module 4, we learned that the ArrayList class in Java has its own resizing policy for addition (insertion) and add operation runs in amortized constant time but its removal is not that efficient.

The ArrayList has to deal with shifting elements every time the `remove(int index)` method is called (except it is the last element).

Let's quickly try to take a look at some running times.

Download lab1.zip file from Canvas and find **ArrayListDemo.java** and **Stopwatch.java** and run **ArrayListDemo.java** on your machine.

What is the running time of adding 100,000 integers?

What is the running time of removing 100,000 integers?

Now, increase the value of size variable to be 1,000,000 and run it again.

What is the running time of addition?

What is the running time of removal?

Now, comment the `tmp.remove(0)` and uncomment `tmp.remove(tmp.size()/2)` or `tmp.remove(tmp.size()-1)` and run it again.

What is the running time of addition?

What is the running time of removal?

Do you see any difference? Can you explain why, if any?

String Manipulation

String manipulation is one of the most common and important operations for algorithms such as parsing text, validating input or data, etc.

There are four major classes in Java in relation to string or character.

- String: *IMMUTABLE* class that work with multiple characters.
- StringBuffer: *MUTABLE* class that work with multiple characters and is synchronized.
- StringBuilder: *MUTABLE* class that work with multiple characters and is not synchronized.
- Character: Wrapper Class whose instances can hold a single character value.

When should you use String and when should you use StringBuffer or StringBuilder?

When you want to prohibit data modification, use String. Otherwise, you may use StringBuffer or StringBuilder class. In most cases, StringBuilder is a good choice.

Problem:

Write a method to perform a basic string compression using the counts of characters.

For example, you can compress the string of "aaaaabbbbcccd" to be "a5b4c3d3". **Another example string is "aabbaabbb" which needs to be compressed as "a2b2a2b3", not "a4b5".**

Your algorithm should also be able to handle the case where compressed string is bigger than the original string. In that case, you should return the original string instead of trying to compress the string. You can assume that an input string has at least two characters and they are all lowercase Alphabet characters which means it does not include any numbers nor special characters.

In the `CompressString1.java` and `CompressString2.java` files we provide on Canvas, try to implement the stubbed sections of each class. Your solutions will be similarly for both java files, with the only difference being the using of `String` vs. `StringBuilder`.

You may want to create a lab1 java project on an IDE of your choice and put the java files into the project.

Once, you are done with implementing, run them and compare the running times.

1. What is the running time of the first class? (Try to run a few times and take note of those times.)
2. Using the Big O notation, this algorithm runs in _____.
(n is the number of characters in a string.)
3. What is the running time of the second class? (Try to run a few times take note of those times.)
4. Using the Big O notation, this algorithm runs in _____.
(n is the number of characters in a string.)
5. Based on the information given in this lab, can you find out why there is the difference in its actual running time, if any?

Place `CompressString1.java` and `CompressString2.java` files into a zip file called `lab1.zip`.

Do not change the file names and make sure to remove all the print statements you used for debugging. Also, make sure to remove any package statements.

Submit “lab1.zip” file on Autolab (<https://autolab.andrew.cmu.edu>).

Do not submit on Canvas unless you are instructed by one of the teaching assistants or the instructor. The questions in this handout are for you to think about. Please do not submit this handout.